Project 00 — Linux + Git + GitHub

The Foundation of Modern Software Engineering

Summary: This document defines the subject matter for Project 00: Linux terminal fundamentals, command-line file manipulation, Git version control workflows, GitHub remote hosting, and SSH key authentication.

Version: 1.0

Contents

I	Instructions	2
II	AI & Peer-Learning Guidelines	3
III	Foreword	4
IV	Exercise 00: Terminal Navigation & Directories	5
V	Exercise 01: File Manipulation & Text Streams	6
VI	Exercise 02: Build a Public GitHub Repository	7
VII	Exercise 03: First GitHub Project + README & SSH Bounce	8
XIV	Submission and Peer-Evaluation	9

Chapter I: Instructions

- These exercises are carefully arranged in order of difficulty, from easiest to hardest. We will not consider a successfully completed harder exercise if an easier one is not fully functional.
- Ensure that you have the appropriate permissions and correct directory structure for all your files.
- You must follow the exact submission procedures and naming conventions for every exercise.
- Your exercises will be checked and graded by your fellow classmates during peer evaluations.
- Additionally, your exercises may be validated by automated evaluation scripts. The evaluation is rigorous and strict. To avoid unpleasant surprises, be as thorough as possible.
- You must not leave any extraneous files in your directories other than those explicitly specified in the subject.
- Have a question? Ask the peer on your right. If not, try the peer on your left.
- Your reference guides are official documentation (MDN, man, Git docs) and verified technical resources.
- Examine the examples and edge cases carefully. They contain critical details.

— Did you check with your right-side neighbor? —

Chapter II: AI Instructions

The Web Piscine is foundational. Your primary mission is to build solid problem-solving reflexes, genuine technical autonomy, and collaborative peer learning.

In the modern tech ecosystem, generative AI shortcuts are tempting. However, shortcuts prevent the formation of neural connections required for deep engineering intuition.

Key Rules on AI Usage:

- Apply your own reasoning to each exercise before consulting external AI tools.
- Never ask an AI to generate solutions directly for you. You must write every line of code yourself.
- During peer evaluation and exams, you will have to explain every architectural decision and command. If you cannot explain your code, you will receive zero points.
- Good practice: You are confused about a concept like Flexbox or Git staging. You discuss it with your peer for 10 minutes, sketch it on paper, and it clicks.
- Bad practice: Copying code without understanding. You fail the peer defense because you cannot answer conceptual questions.

— Learning is not knowing the answer; it is developing the ability to find it. —

Chapter III: Foreword

"In the beginning was the command line." — Neal Stephenson

Before graphical operating systems, developers interacted with machines using raw text commands. Even today, cloud servers, supercomputers, and modern production deployments run on headless Linux environments.

In this project, you step into the world of Linux and Git. Master the shell, command your machine, and protect your code with version control.

— Google is your friend. —

Chapter IV — Exercise 00: Navigation & Directories

Directory: ex00/

Files to Submit: tree_structure.txt

Authorized: pwd, ls, cd, mkdir, clear

Navigation and structure without a mouse.

- Open your Linux terminal and locate your current directory with pwd.
- Create a project workspace directory named dev_workspace and enter it.
- Create subdirectories projects, archives, and tools in a single command.
- Create nested paths projects/web/frontend and projects/web/backend using mkdir -p.
- Navigate through your folder tree using relative paths (cd ..), verify your path with pwd, then navigate back to dev_workspace.
- Generate a full recursive listing (ls -laR) of dev_workspace and save it into tree_structure.txt inside ex00/.

! No graphical file managers allowed; navigate purely through the terminal.

— Navigation is the first skill of a terminal master. —

Chapter V — Exercise 01: File Manipulation

Directory: ex01/

Files to Submit: manage_files.sh, run_log.txt

Authorized: touch, echo, cat, cp, mv, rm

Commanding files and text streams.

- Write an automated shell script named manage_files.sh.
- Create draft.txt using touch, then write to it using echo and output redirection (>).
- Append a second line with >> and display the file contents in terminal with cat.
- Copy draft.txt to draft_backup.txt using cp, then rename it to final_report.txt with mv.
- Create a temporary file and cleanly remove it using rm.
- Make your script executable (chmod +x), execute it, and save the terminal output into run_log.txt.

! Be careful with > versus >>. Overwriting data unintentionally cannot be undone.

— Automate what you repeat. —

Chapter VI — Exercise 02: GitHub Repository

Directory: ex02/

Files to Submit: repo_url.txt, git_log.txt

Authorized: git init, status, add, commit, log, remote, push

Connecting your machine to the cloud.

- Configure your global Git username and email.
- Initialize a local Git repository in ex02/ using git init.
- Create a file hello.txt, stage it with git add, and commit it with a clear commit message.
- Create a public repository on GitHub named discovery-web-piscine.
- Link your local repo to GitHub using git remote add origin and push to main.
- Save your public GitHub repository URL in repo_url.txt and git log output in git_log.txt.

— Commit early, commit often. —

Chapter VII — Exercise 03: Project & README

Directory: ex03/

Files to Submit: README.md, git_workflow.txt, id_ed25519_pub.txt (Bonus)

Authorized: git, ssh-keygen, Markdown

Professional documentation & secure authentication.

- Create a comprehensive, well-structured README.md in Markdown.
- Your README must feature a section titled "Why Git Makes Development Easier", explaining in your own words how version control safeguards code, tracks history, and enables team collaboration.
- Record the complete edit-add-commit-push cycle in git_workflow.txt.
- Bounce / Bonus Requirement: Generate an ed25519 SSH key pair, add the public key to GitHub, convert your remote origin URL to SSH (git@github.com:...), and push your changes securely without typing passwords.

! Never commit or submit your private key! Only submit id_ed25519_pub.txt for the bonus.

— Security first: private keys stay strictly private. —

Chapter XIV: Submission & Evaluation

- Submit your assignment to your Git repository as instructed. Only the work pushed to your repository branch will be evaluated during defense.
- Double-check your directory structure, file permissions, and exact file names before final submission.

! You must submit only the files explicitly required by the project instructions.

- Prepare to demonstrate your code live, explain your logic line-by-line, and answer questions about core concepts.

— RTFM and verify your work before peer defense! —